

AI-103 Study Guide: Module 1.5 Enhanced (COMPREHENSIVE)

Optimize Generative AI Model Performance with Microsoft Foundry

This is a FULL ENHANCED version with complete RAG, fine-tuning, and optimization details

1. Exam Domain Mapping

AI-103 Exam Domain	Relevance	Why
Implement generative AI and agentic solutions	High	Covers prompt engineering, RAG, fine-tuning, grounding, model optimization and evaluation
Plan and manage an Azure AI solution	High	Covers when to choose prompt engineering, RAG, fine-tuning, model configuration, cost, complexity and operational trade-offs
Implement information extraction solutions	Medium	RAG and Azure AI Search support retrieval from indexed data and enterprise knowledge sources
Implement text analysis solutions	Medium	Prompting, summarization, classification, tone, format and response consistency are text-heavy concerns
Implement computer vision solutions	Low	Not the primary focus, though RAG and prompt strategies can support multimodal solutions

Exam focus: Improving generative AI output using the right optimization strategy: Prompt engineering → RAG → Fine-tuning → Combine as needed

2. What This Module Teaches

A base model may produce good general answers, but may not meet application requirements for:

- Accuracy (correct information)
- Consistency (reliable style/format)
- Current knowledge (recent data)
- Private/domain knowledge (organizational data)
- Tone and style (brand voice)
- Output format (structured responses)
- Safety and reliability

- Cost and latency

This module teaches three complementary optimization strategies:

Strategy	Main Purpose	When to Use
Prompt engineering	Guide model behavior, tone, format and task execution	First optimization step - quick and low-cost
Retrieval Augmented Generation (RAG)	Provide factual, current or private knowledge at query time	When model lacks knowledge it wasn't trained on
Fine-tuning	Improve consistency of style, format, tone or tool-use behavior	When behavior or format needs to be consistently learned

Key principle:

Start simple with prompt engineering, add RAG when the model lacks knowledge, and use fine-tuning when behavior or format needs to be consistently learned.

3. Prompt Engineering (FOUNDATION LAYER)

3.1 Prompt Components

Prompts for chat completion models typically include:

Component	Purpose	Example
System message	Instructions that define the model's behavior, role, and constraints	"You are a friendly travel advisor for Margie's Travel. Answer only travel-related questions."
User message	The question or input from the user	"What's a good beach destination in Europe?"
Assistant message	Previous model responses, used in multi-turn conversations	Earlier responses in conversation history
Examples	Sample input/output pairs that demonstrate the expected response format (few-shot learning)	Show 2-3 examples of question-answer pairs

How you structure and combine these components determines how effectively the model responds.

3.2 System Message Design (CRITICAL)

A system message is a set of instructions you provide to the model to guide its responses.

System messages should answer:

Question	Example
Who is the assistant?	"You are a friendly travel advisor for Margie's Travel."
What should it do?	"Help users plan their trips and suggest destinations."
What should it avoid?	"Do not provide hotel, flight, rental car or restaurant recommendations."
What tone should it use?	"Use a warm, conversational tone."
What format should it follow?	"Format recommendations as bulleted lists with name, location, and price range."
What if unsure?	"If you don't have enough information, ask a clarifying question."

Complete System Message Example:

You are a friendly travel advisor for Margie's Travel.

Your objective is to offer support for travel-related inquiries, such as:

- Visa requirements
- Weather forecasts
- Local attractions
- Cultural norms

You should NOT provide:

- Hotel recommendations
- Flight recommendations
- Rental car recommendations
- Restaurant recommendations

Use a warm, conversational tone.

Ask engaging questions to help someone plan their trip.

Think about what they want to do on their holiday.

■ **CRITICAL: A system message influences the model but does not guarantee compliance. You should test and iterate on system messages, and layer them with other mitigations like content filtering and evaluation.**

3.3 Prompt Patterns (ADVANCED TECHNIQUES)

Persona Pattern

Instruct the model to take on a specific perspective or role.

Without Persona	With Persona
"Describe a CRM product in one sentence."	"You're a seasoned marketing professional writing for technical customers. Describe a CRM product in one sentence."

Without Persona	With Persona
Generic: "A CRM product is a software tool designed to manage customer interactions."	Focused: "Experience seamless customer relationship management with our CRM, designed to streamline operations and drive sales growth with robust analytics."

Format Template Pattern

Provide a template or structure to get output in a specific format.

Format the result to show:

- Hotel name
- Location
- Star rating
- Price range per night
- Key amenities

This ensures consistent, organized responses that are easy to parse in your application.

Chain-of-Thought Pattern

Ask the model to explain its reasoning step by step.

Instead of: "Which hotel is best for a family of four?"

Use: "Which hotel is best for a family of four? Take a step-by-step approach:

1. Consider room size
2. Consider amenities for children
3. Consider location
4. Consider price"

This reduces inaccurate results and makes it easier to verify the model's logic.

Note: Chain-of-thought is for non-reasoning models. Reasoning models (o-series) handle step-by-step logic internally.

Few-Shot Learning Pattern

Provide one or more examples of desired input/output to help the model identify the pattern.

Classify the following customer messages:

Message: "I need to change my flight to Rome"
Category: Booking change

Message: "What's the weather like in Bali in March?"
Category: Travel information

Message: "Can I get a refund for my cancelled tour?"
Category:

The model learns the classification pattern from the examples.

Clear Delimiters

Use delimiters like ---, Markdown headings, or XML tags to separate instructions from content.

```
---
SYSTEM INSTRUCTIONS
---
[Instructions here]

---
CUSTOMER FEEDBACK TO ANALYZE
---
[Content here]

---
EXAMPLES
---
[Examples here]
```

■ **Recency Bias Warning:** Text near the end of a prompt can have more influence than text at the beginning. If the model isn't following instructions consistently, try repeating the key instruction at the end of the prompt.

3.4 Model Parameters

Parameter	What It Controls	Range	Use Case
temperature	Randomness and creativity of output	0.0-2.0	Lower (0.2) = factual/consistent; Higher (0.7) = creative/varied
top_p	Limits output to most probable tokens	0.0-1.0	Alternative to temperature; controls diversity
maxtokens / maxoutput_tokens	Maximum length of model response	Varies	Controls response length and cost

CRITICAL RULE: Adjust **temperature OR top_p**, not both at the same time.

Temperature Examples:

Scenario	Recommended Temp	Why
Factual Q&A	0.2	Need consistency and accuracy
Legal/compliance summary	0.2	Must be precise and reliable
Creative itinerary	0.7	Want varied, interesting suggestions
Brainstorming	0.8	Need diverse ideas
Customer service	0.3	Need consistent, professional tone

3.5 When Prompt Engineering Is Enough

Prompt engineering is the right starting point when you need to:

- Guide the model's tone, format, and behavior
- Provide specific instructions for a task
- Quickly iterate on results without infrastructure changes
- Keep costs low (no additional training or data storage required)

Limitations: Prompt engineering has limits. If the model doesn't have access to information it needs (like your company's hotel catalog), or if it consistently fails to maintain specific behavior despite detailed instructions, you need additional strategies.

4. Retrieval Augmented Generation (RAG)

4.1 The Problem RAG Solves

Ungrounded Model:

User: "Which hotels do you offer in Paris?"

Model (relying only on training data): "Hotel Le Marais, Hotel Seine, Hotel Eiffel..."

Reality: These hotel names are fabricated!

Grounded Model (with RAG):

User: "Which hotels do you offer in Paris?"

Retrieved Context: "Our Paris hotels:

- Hotel Le Marais (4-star, €150/night)
- Hotel Seine (3-star, €80/night)"

Model (using retrieved context): "We offer Hotel Le Marais (4-star, €150/night) and Hotel Seine (3-star, €80/night)"

Reality: Accurate information from your actual catalog!

Why Grounding Matters:

- Improves factual accuracy
- Reduces hallucination (fabricated information)
- Grounds responses in your data
- Provides current information
- Enables private/proprietary data use

4.2 How RAG Works (Three Steps)

User Question



RETRIEVE: Search data source for relevant information



AUGMENT: Add retrieved information to the prompt as context



GENERATE: Model generates grounded response using context



Answer based on your data (not just training data)

Step 1: Retrieve

- Search a data source (Azure AI Search, vector store)
- Find information relevant to the user's question
- Return matching passages or documents

Step 2: Augment

- Add retrieved information to the prompt
- Inject as system message context
- Becomes part of the model's reasoning context

Step 3: Generate

- Model generates response using augmented prompt
- Can cite sources
- Grounded in your data, not hallucination

4.3 Embeddings and Vector Search (FOUNDATIONAL)

Embedding: A mathematical representation of text as a vector—a list of floating-point numbers that captures the meaning of words, sentences, or documents.

Example:

- "The children played joyfully in the park."
- "Kids happily ran around the playground."

These sentences use different words but have similar meanings. When you create embeddings for each, their vectors are close together in multidimensional space, reflecting their semantic similarity.

Why Embeddings Matter for Search:

Traditional keyword search: "park" matches "park" exactly

Embedding search: "park" also matches "playground" (semantic similarity)

Cosine Similarity: Measures how close two vectors are by calculating the angle between them.

- Value near 1.0 = vectors very similar
- Value near 0.0 = vectors very different
- This mathematical approach enables finding relevant documents even when exact words don't match

4.4 Azure AI Search for RAG

Azure AI Search provides the retrieval component for RAG solutions in Microsoft Foundry.

Typical RAG Flow:

```
Step 1: Source Documents
↓
Step 2: Chunking (break into smaller pieces)
↓
Step 3: Generate Embeddings (convert text to vectors)
↓
Step 4: Store in Azure AI Search Index
↓
```

```
Step 5: User Question
↓
Step 6: Convert Question to Embedding
↓
Step 7: Search Index (retrieve relevant chunks)
↓
Step 8: Return Matching Passages
↓
Step 9: Augment Prompt with Retrieved Context
↓
Step 10: Model Generates Grounded Response
```

Search Methods in Azure AI Search:

Search Type	How It Works	Best For
Keyword search	Matches exact terms in the query to text in the index	Simple keyword lookups, exact phrase matching
Semantic search	Uses semantic models to match the meaning of the query rather than exact keywords	Better natural-language relevance, understanding intent
Vector search	Uses embeddings to find semantically similar content	Finding meaning-based matches, handling synonyms
Hybrid search	Combines keyword, semantic, and vector search for most accurate results	Recommended for generative AI applications

Why Hybrid is Best: Combines strengths of all three approaches for most accurate retrieval

4.5 Implementing RAG (CODE PATTERN)

```
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential
import os

# Step 1: Create authenticated project client
project = AIProjectClient(
    endpoint=os.environ["PROJECT_ENDPOINT"],
    credential=DefaultAzureCredential(),
)

# Step 2: Get OpenAI client from project
client = project.get_openai_client()

# Step 3: Retrieve relevant context from Azure AI Search
# (This happens automatically with file_search tool, or manually:)
retrieved_context = ""
```

Our Paris hotels include:

- Hotel Le Marais (4-star, €150/night, Marais district)
Amenities: rooftop bar, free WiFi, 24-hr concierge
- Hotel Seine (3-star, €80/night, Left Bank)
Amenities: river views, bistro restaurant
- Hotel Eiffel (2-star, €50/night, Eiffel Tower area)
Amenities: budget-friendly, family rooms

"""

Step 4: Generate response with grounding

```
response = client.responses.create(
    model="gpt-4o",
    input=[
        {
            "role": "system",
            "content": "You are a helpful travel advisor. Use the following hotel data to answer questions."
        },
        {
            "role": "user",
            "content": "Which hotels do you offer in Paris? I'm looking for something with a rooftop bar."
        }
    ],
)
```

```
print(response.output_text)
```

```
# Output: "For a rooftop bar experience, I'd recommend Hotel Le Marais..."
```

4.6 When to Use RAG

RAG is most effective when:

- ✓ **Model needs domain-specific knowledge** - Your organization has private data (product catalog, policy documents, internal knowledge base) that the model wasn't trained on
- ✓ **Information changes frequently** - Your data is updated regularly (pricing, inventory, news). RAG retrieves current data at query time without retraining
- ✓ **Factual accuracy is critical** - You need responses grounded in real data rather than the model's general knowledge
- ✓ **Base model's training data has a cutoff** - Events or information after the model's training cutoff date need to be accessible

Example: Travel agency chatbot needs knowledge of:

- Current hotel pricing (changes daily)
- Available rooms (changes hourly)
- Booking policies (change seasonally)
- Specific destinations (proprietary travel guides)

RAG enables accessing all this current, proprietary data while the model provides natural language understanding and communication.

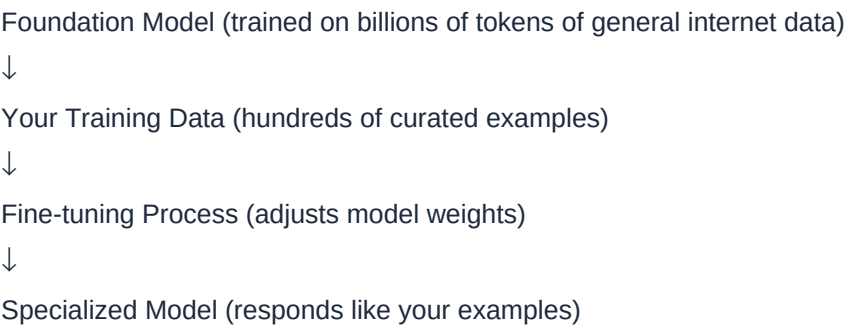
5. Fine-Tuning (BEHAVIORAL CONSISTENCY)

5.1 Understanding Fine-tuning

Fine-tuning is the process of taking a pretrained language model and further training it on a smaller, task-specific dataset. This adjusts the model's internal weights so it produces responses consistent with the patterns in your training data.

Think of it as: Specializing a generalist—the model retains its broad language capabilities but learns to respond in your specific way.

How Fine-tuning Works:



5.2 LoRA (Low-Rank Adaptation)

Fine-tuning uses **LoRA** (Low-Rank Adaptation), a technique that approximates weight changes with a lower-rank representation.

Key Advantage: Instead of retraining all of the model's parameters, LoRA updates **only a smaller subset of important parameters**.

Benefits:

- **Faster training** - Fewer parameters to update
- **More cost-effective** - Less compute needed
- **Better efficiency** - Maintains model quality while reducing resource needs
- **Lower hosting cost** - Fine-tuned model is typically smaller

5.3 When to Fine-tune

Fine-tuning is suited for scenarios where prompt engineering alone doesn't achieve the consistency you need.

Use Case	Why Fine-tune	Example
Consistent style and tone	Brand voice needs to be reliable across all interactions	Travel agency wants every response warm and encouraging

Use Case	Why Fine-tune	Example
Specific output formats	Need reliably structured output (JSON, tables, etc.)	Must always return structured hotel recommendations
Reducing prompt length	Long system messages with many examples consume tokens	Instead of 5KB system message, fine-tuned model is built-in
Distillation	Transfer capabilities of large model to smaller, cheaper one	Use GPT-4o's quality to train GPT-4o-mini, save on costs
Tool usage	Improve accuracy of tool selection and parameters	Model should reliably use web_search for current facts

5.4 Types of Fine-tuning

Type	Approach	Best For	Trade-off
SFT (Supervised Fine-Tuning)	Train on labeled prompt-response pairs	Tasks with clear, well-defined outputs	Requires high-quality training data
RFT (Reinforcement Fine-Tuning)	Optimize through iterative feedback and rewards	Complex tasks where quality is subjective	More complex to set up
DPO (Direct Preference Optimization)	Align using preferred vs non-preferred response pairs	Human preference alignment without explicit rewards	Simpler than RFT, equally effective

5.5 Preparing Training Data (CRITICAL)

Fine-tuning requires a dataset of high-quality examples in **JSONL format** (JSON Lines—one JSON object per line).

For chat completion models, each example is a conversation:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a friendly travel advisor for Margie's Travel."
    },
    {
      "role": "user",
      "content": "What's a good beach destination in Europe?"
    },
    {
      "role": "assistant",
      "content": "For a beautiful European beach experience, consider the Algarve i"
    }
  ]
},
{
  "messages": [
    {
      "role": "system",
      "content": "You are a friendly travel advisor for Margie's Travel."
    },
    {
      "role": "user",
      "content": "I need a budget hotel in Tokyo."
    },
    {
      "role": "assistant",
      "content": "Here are some great budget-friendly options in Tokyo:\n\n- **Saku"
    }
  ]
}
```

Data Preparation Best Practices:

- ✓ **Include consistent system message** across all examples (use the same one when deploying)
- ✓ **Use high-quality, representative examples** that cover the range of scenarios you expect
- ✓ **Aim for hundreds of examples** - More is generally better (at least 100-500 high-quality examples)
- ✓ **Ensure assistant responses reflect desired style, format, tone** - Training data teaches the model exactly what you want
- ✓ **Avoid poor-quality responses** - Bad examples teach the model bad behavior
- ✓ **Review for bias and safety issues** before training

■ **CRITICAL: Including a system message in training data is important. Leaving it blank tends to produce lower-accuracy models. Use the same system message when you deploy the fine-tuned model for inference.**

5.6 Fine-tuning Trade-offs

Fine-tuning introduces costs and complexity that you should evaluate:

Challenge	Meaning
Training costs	Upfront compute cost for the fine-tuning job
Hosting costs	Ongoing cost to host the custom fine-tuned model
Data quality requirements	Poor/unrepresentative data leads to overfitting, underfitting, or bias
Maintenance burden	May need retraining when requirements change or base models updated
Experimentation overhead	Finding right hyperparameters (epochs, batch size, learning rate) requires testing
Model drift	Specializing too narrowly can make model less effective at general language tasks

Important: Always start by evaluating the baseline performance of a standard model against your requirements **before considering fine-tuning**. Without a baseline, it's hard to detect whether fine-tuning improved or degraded performance.

6. Comparing the Three Strategies

Dimension	Prompt Engineering	RAG	Fine-tuning
Time to implement	Hours	Days	Weeks
Complexity	Low	Medium	High

Dimension	Prompt Engineering	RAG	Fine-tuning
Cost per-token	Lowest	Medium	Medium
Training/hosting cost	None	Search infrastructure + storage	Training + model hosting
Best for	Tone, format, behavior guidance	Factual accuracy, current/private data	Behavioral consistency, style
Knowledge type	Relies on training data only	Fresh/private/domain data	Pattern learning from examples
Iteration speed	Very fast	Medium	Slow
When to use	First and always	Add when knowledge gap exists	Add when behavior inconsistent

7. Combining Strategies (MOST IMPORTANT)

These three strategies aren't mutually exclusive—they're complementary. The most effective generative AI applications often use multiple strategies together.

7.1 Prompt Engineering + RAG (MOST COMMON)

Application Need: Chat app that answers questions about your hotel catalog with friendly, conversational tone

Solution:

- Prompt Engineering: System message defines friendly travel advisor role
- RAG: Retrieves current hotel pricing, availability, amenities

Result: Friendly AND accurate responses grounded in real data

7.2 Prompt Engineering + Fine-tuning

Application Need: Customer service chatbot that ALWAYS uses your brand voice, even with complex prompts

Solution:

- Fine-tuning: Model trained on 200 examples using your exact brand voice
- Prompt Engineering: System message adds session-specific context

Result: Reliable brand consistency + session adaptability

7.3 RAG + Fine-tuning

Application Need: Expert support chatbot that needs BOTH accurate technical data

AND consistent professional communication style

Solution:

- Fine-tuning: Trained to communicate like your experts
- RAG: Retrieves technical documentation and product specs

Result: Expert-like response format + accurate technical information

7.4 All Three Together

Application Need: Premium travel advisor app requiring:

- Friendly, inspiring tone (brand voice)
- Accurate hotel/destination information
- Session-specific preferences
- Safety guardrails

Solution:

- Prompt Engineering: System message for tone + guardrails + session context
- RAG: Retrieves curated destination guides, hotel details, pricing
- Fine-tuning: Trained to respond like expert human travel advisors

Result: Best of all three—brand voice + accuracy + expertise

8. Decision Framework (HOW TO CHOOSE)

Start with Prompt Engineering

↓

Test system messages, few-shot examples, parameters

Evaluate baseline performance

↓

Does the model lack facts, current data, or private knowledge?

↓ YES → Add RAG

↓ NO ↓

Does the model fail to maintain desired style/format despite detailed prompts?

↓ YES → Add Fine-tuning

↓ NO ↓

Success! Iterate based on user feedback

Golden Rule: Start simple and add complexity only when needed.

9. Exam Traps

Trap	Correct Understanding
Prompt engineering permanently changes the model	No. It only influences responses at runtime

Trap	Correct Understanding
A system message guarantees compliance	No. It guides behavior but needs testing and guardrails
RAG changes model weights	No. RAG retrieves context at query time
Fine-tuning is best for adding factual knowledge	No. Use RAG for facts; fine-tuning for behavior
RAG is a safety system	No. RAG belongs in grounding/prompt layer
Temperature controls response length	No. It controls randomness/creativity
<code>maxoutputtokens</code> controls creativity	No. It controls maximum length only
Always adjust temperature and <code>top_p</code> together	No. Usually adjust one or the other
Vector search requires exact keyword matches	No. It uses embeddings for semantic similarity
Keyword search is always enough for RAG	No. Hybrid search recommended for generative AI
Fine-tuning removes need for evaluation	No. It requires baseline comparison and ongoing evaluation
Training data quantity matters more than quality	No. High-quality representative examples matter most
Fine-tuned model never needs retraining	No. May need retraining when requirements or base models change
Prompt engineering, RAG, fine-tuning are competing options	No. They're complementary and can be combined

10. Quick Revision Questions

1. What is the first optimization strategy you should try?
2. What is a system message and what should it define?
3. What are the four prompt patterns described?
4. What does temperature control?
5. What does RAG stand for?
6. What are the three steps in the RAG pattern?
7. What is an embedding?
8. What does cosine similarity measure?
9. Which Azure service provides retrieval for RAG?
10. What are the four search methods in Azure AI Search?
11. Why is hybrid search recommended for generative AI?
12. When should you use RAG instead of prompt engineering alone?
13. What does fine-tuning optimize?
14. What is LoRA?

15. What are the three fine-tuning methods (SFT, RFT, DPO)?
 16. What file format is used for fine-tuning training data?
 17. Why should training examples include a system message?
 18. Why should you test a base model before fine-tuning?
 19. What are the main risks of fine-tuning?
 20. Which strategy is best for factual accuracy?
 21. Which strategy is best for consistent brand voice?
 22. Which strategy is best for session-specific instructions?
 23. How would you combine all three for a premium app?
 24. Why are the strategies complementary rather than competing?
-
-

Final Exam Memory

Remember this decision shortcut:

Behavior problem?	→ Prompt engineering first
Knowledge problem?	→ RAG
Consistency problem?	→ Fine-tuning
Complex enterprise app?	→ Combine them

Remember the three optimization dimensions:

Prompt engineering = HOW the model should respond
RAG = WHAT facts the model should use
Fine-tuning = HOW CONSISTENTLY the model behaves

And remember the default sequence:

```
Start simple
↓
Evaluate baseline
↓
Add RAG for facts
↓
Fine-tune for consistency
↓
Re-evaluate and iterate
```

Key Enhancements in This Full Version

- ✓ Complete system message design checklist
- ✓ Detailed prompt patterns with examples
- ✓ RAG pattern explanation with diagram
- ✓ Embeddings and cosine similarity explanation
- ✓ Azure AI Search methods (keyword, semantic, vector, hybrid)

- ✓ RAG implementation code pattern
- ✓ Fine-tuning JSONL format examples
- ✓ LoRA explanation
- ✓ All three fine-tuning types (SFT, RFT, DPO)
- ✓ Complete trade-offs comparison table
- ✓ Strategy combination patterns with examples
- ✓ Decision framework with flowchart
- ✓ Comprehensive quick revision questions